1 MATLAB 개요

MATLAB Programming

Jin Hee Yoon
Dept. of Artificial Intelligence and Information Technology
Sejong University

Spring, 2026

Chapter Overview

- MATLAB 소개
- 설치 및 기본 화면
- 기본 계산과 Command Window
- 배열과 그래프의 기초
- 스크립트, 함수, 도움말

MATLAB 소개

MATLAB이란 무엇인가?

- MATLAB은 **MATrix LABoratory**의 약자로, 행렬 기반의 수치 계산과 데이터 분석을 위한 소프트웨어이자 프로그래밍 언어이다.
- 수학, 통계, 공학, 제어, 신호처리, 최적화, 인공지능 등 다양한 분야에서 널리 사용된다.
- 계산, 시각화, 알고리즘 개발, 프로토타이핑을 하나의 환경에서 수행할 수 있다.
- 특히 벡터와 행렬 연산을 자연스럽게 표현할 수 있다는 점이 큰 장점이다.

왜 MATLAB을 배우는가?

- 수학적 표현과 코드가 비교적 가깝다.
- 벡터와 행렬 계산을 직관적으로 수행할 수 있다.
- 그래프를 빠르게 그릴 수 있어 결과 해석이 쉽다.
- 공학 및 과학 계산에 필요한 함수와 Toolbox가 풍부하다.
- 학습용 실습부터 연구 및 산업 현장 모델링까지 널리 사용된다.

MATLAB의 역사

- 1970년대 후반, Cleve Moler가 학생들이 수치해석 라이브러리를 쉽게 사용할 수 있도록 초기 MATLAB을 개발하였다.
- 이후 Jack Little과 함께 상용 환경으로 발전하였고, **MathWorks**가 설립되었다.
- 오늘날 MATLAB은 단순한 계산 도구를 넘어 데이터 분석, 시뮬레이션, 제어, AI까지 지원하는 통합 플랫폼으로 발전하였다.

MATLAB의 주요 특징

1. **행렬 중심 언어**
2. **간결한 문법**
3. **강력한 시각화**
4. **풍부한 Toolbox**
5. **Simulink와의 연동**

- 따라서 MATLAB은 빠른 계산 실험과 공학적 모델링에 매우 적합하다.

MATLAB의 활용 분야

공학 분야

- 제어 시스템
- 신호처리
- 통신
- 로봇공학
- 전력 시스템

데이터/AI 분야

- 통계 분석
- 머신러닝
- 딥러닝
- 최적화
- 데이터 시각화

MATLAB과 다른 언어 비교

구분	MATLAB	Python	C
학습 난이도	비교적 쉬움	보통	상대적으로 높음
행렬 연산	매우 강함	라이브러리 필요	직접 구현 필요
시각화	매우 편리	강력함	제한적
실행 속도	보통	보통	빠름
공학용 확장성	매우 풍부	매우 다양	제한적

설치 및 기본 화면

MATLAB 설치 및 실행 환경

- MATLAB은 **MathWorks**에서 제공하는 상용 소프트웨어이다.
- 학생용, 개인용, 학교 라이선스(Campus License) 등 다양한 방식으로 사용할 수 있다.
- Windows, macOS, Linux 환경에서 설치 가능하다.
- 보통 학교 계정 또는 개인 MathWorks 계정으로 로그인하여 설치한다.

설치 과정의 기본 흐름

1. MathWorks 계정 생성 또는 학교 라이선스 확인
2. 설치 파일 다운로드
3. 라이선스 선택
4. 필요한 Toolbox 선택
5. 설치 후 MATLAB 실행

참고
기본 실습은 기본 MATLAB만으로 충분하며, 이후 필요에 따라 Toolbox를 사용할 수 있다.

MATLAB 기본 화면 구성

- **Command Window**: 명령을 직접 입력하고 즉시 실행
- **Editor**: 스크립트와 함수 파일 작성
- **Current Folder**: 현재 작업 폴더 확인
- **Workspace**: 현재 생성된 변수 목록 확인
- **Command History**: 이전 명령 기록 확인

핵심
MATLAB은 **명령창에서 바로 실행하는 방식**과 **파일로 저장하여 실행하는 방식**을 모두 지원한다.

Command Window와 Editor

Command Window

- 한 줄씩 직접 입력
- 즉시 실행
- 짧은 계산 및 테스트에 적합

Editor

- 여러 줄 코드를 작성
- `.m` 파일로 저장
- 반복 실행과 재사용에 적합

기본 계산과 Command Window

명령창 사용의 기초

- 명령을 입력하고 Enter를 누르면 즉시 실행된다.
- 위/아래 화살표로 이전 명령을 다시 불러올 수 있다.
- 계산 결과는 필요에 따라 ans에 임시 저장되기도 한다.

```
>> 8/10
ans =
    0.8000

>> 5*ans
ans =
    4
```

기본 4칙연산

연산	기호
덧셈	+
뺄셈	-
곱셈	*
나눗셈	/
거듭제곱	^

$$8+3, \qquad 10-4, \qquad 5*6, \qquad 8/10, \qquad 2^3$$

세미콜론, 주석, 변수 확인

- ; : 출력 생략
- % : 주석
- who, whos : 변수 확인
- clear, clc : 변수 삭제 및 화면 정리

```
a = 10;       % 변수 a
b = 20;       % 변수 b
c = a + b

who
whos
```

연산자 우선순위

- MATLAB도 일반적인 수학 규칙처럼 연산자 우선순위를 따른다.
- 괄호를 사용하면 계산 순서를 명확히 할 수 있다.

```
8 + 3*5
(8 + 3)*5

3*4^2 + 5
(3*4)^2 + 5
```

아주 간단한 내장 함수

함수	의미
`sqrt(x)`	제곱근
`abs(x)`	절댓값
`exp(x)`	지수함수
`log(x)`	자연로그
`sin(x), cos(x)`	삼각함수

```
sqrt(25)
abs(-7)
exp(1)
sin(pi/2)
cos(0)
```

특수 상수와 표시 형식

```
pi
1/0
0/0

format short
format long
```

- `pi`: 원주율
- `Inf`: 무한대
- `NaN`: 정의되지 않은 수
- `format`: 출력 형식 조정

MATLAB 변수 이름 규칙

MATLAB에서 변수(variable)는 데이터를 저장하기 위한 이름이다.

변수 이름 작성 규칙

- 반드시 **문자(letter)**로 시작해야 한다.
- 이후에는 **문자, 숫자, 밑줄(_)**을 사용할 수 있다.
- 공백(space)은 사용할 수 없다.
- MATLAB은 **대소문자를 구분(case-sensitive)**한다.
- 변수 이름의 최대 길이는 약 **63자**이다.

올바른 변수 이름 예

- `x`
- `velocity`
- `temperature1`
- `data_value`

잘못된 변수 이름의 예

다음과 같은 이름은 MATLAB 변수로 사용할 수 없다.

잘못된 변수 이름

- `1value` (숫자로 시작)
- `my data` (공백 포함)
- `a-b` (연산 기호 사용)
- `for`, `while`, `if` (MATLAB 예약어)

좋은 변수 이름 작성 방법

- 변수의 의미가 드러나는 이름을 사용하는 것이 좋다.
- 예: `v`, `velocity`, `avg_temperature`
- 코드의 가독성과 유지보수성이 크게 향상된다.

배열과 그래프의 기초

배열(array)의 개념

- MATLAB의 핵심 자료구조는 **배열**이다.
- 스칼라, 벡터, 행렬을 모두 배열의 관점에서 다룰 수 있다.
- 하나의 명령으로 배열 전체 원소에 대한 계산을 수행할 수 있다.

벡터와 행렬 생성

```
x = [1 2 3 4]
y = [1; 2; 3; 4]

A = [1 2 3; 4 5 6; 7 8 9]
```

- 같은 행의 원소는 공백 또는 쉼표로 구분한다.
- 행을 바꿀 때는 세미콜론 ;을 사용한다.

콜론 연산자를 이용한 배열 생성

```
u = 0:0.1:10
x = 1:5
y = 0:2:10
```

$$1:5 \Rightarrow [1\ 2\ 3\ 4\ 5]$$

$$0:2:10 \Rightarrow [0\ 2\ 4\ 6\ 8\ 10]$$

- 형식은 시작값:간격:끝값이다.

자주 쓰는 배열 생성 함수

```
zeros(2,3)
ones(3,2)
eye(3)
rand(2,2)
length(u)
```

- `zeros`: 영행렬
- `ones`: 1로 채운 배열
- `eye`: 단위행렬
- `rand`: 난수 배열

배열 인덱싱

```
u = 0:0.1:10;
u(7)

A = [1 2; 3 4];
A(2,1)
```

- MATLAB의 인덱스는 **1부터 시작**한다.
- 이는 Python, C와 다른 중요한 특징이다.

그래프를 왜 그리는가?

- 계산 결과를 수치만으로 보는 것보다 그래프로 보면 훨씬 직관적이다.
- 함수의 모양, 추세, 주기성 등을 빠르게 파악할 수 있다.
- MATLAB은 배열을 만들고, 계산하고, 바로 plot하는 흐름이 매우 편리하다.

아주 간단한 그래프 예제

```
x = 0:0.01:7;
y = 3*cos(2*x);

plot(x,y)
xlabel('x')
ylabel('y')
grid on
```

배열과 그래프의 연결

```
u = 0:0.1:10;
w = 5*sin(u);

plot(u,w)
xlabel('u')
ylabel('w = 5 sin(u)')
grid on
```

- 먼저 배열을 만들고,
- 배열의 각 원소에 대해 계산한 뒤,
- 그 결과를 시각화하는 것이 MATLAB의 대표적인 사용 흐름이다.

스크립트, 함수, 도움말

두 가지 실행 방식

- **대화형 모드**: Command Window에 직접 입력
- **파일 실행 모드**: `.m` 파일에 저장 후 실행

구분
짧은 테스트는 Command Window, 반복 실행과 정리는 Script/Function이 더 적합하다.

MATLAB의 m-파일

MATLAB 프로그램은 보통 `.m` 확장자를 가지는 파일에 저장된다.

m-file의 두 가지 종류

- Script 파일
- Function 파일

- m-파일 안에 반드시 함수를 정의해야 하는 것은 아니다.
- 단순히 명령어만 작성된 m-파일도 존재한다.

Script의 기초

Script 파일은 원래는 **함수 정의 없이 명령어만 포함된 m-파일**이다.

```
% myscript.m
a = 3;
b = 4;
c = a^2 + b^2;
disp(c)
```

- Script는 여러 줄의 명령을 순서대로 실행하는 `.m` 파일이다.

Script 파일

다음 script 이름을 `plot_example.m`으로 저장했다고 하면

예

```
clc; clear;

x = -10:0.1:10;
y = x.^2;

plot(x,y)
grid on
```

실행 방법

command window 에서

```
>> plot_example
```

Script 파일의 특징

- 원래는 `function` 키워드가 없다 (최근 버전 파일 내 실행으로 사용가능)
- 입력값과 출력값 개념이 없다
- Command Window와 **같은 workspace**를 사용한다
- 계산 실험이나 그래프 작성에 자주 사용된다

Function 파일

Function 파일은 **함수를 정의하는 m-파일**이다. 사용자 정의 함수는 반드시 m 파일 안에서 정의하고 command window 에서 호출해서 사용한다.

```
function 출력변수 = 함수이름(입력변수)
    함수 내용
end
```

예:

```
function y = square_num(x)
y = x^2;
end
```

호출:

```
>>square_num(5)
```

Function 파일의 특징

- `function` 키워드로 시작한다
- 입력과 출력이 존재한다
- Script와 달리 **독립된 workspace**를 사용한다
- 재사용 가능한 계산 기능을 만들 때 사용한다

중요한 규칙!!
하나의 m 파일에 여러개의 함수를 정의할 수 있으나 파일 이름과 첫 번째 함수 이름은 동일해야 한다.

Script 안의 Local Function

최근 MATLAB 버전에서는 Script 아래에 **Local Function**을 정의할 수 있다.

Local Function은 Script 파일 내부에서만 사용할 수 있는 함수이다.

- Script 아래에 정의된 함수
- Command Window나 다른 파일에서는 호출할 수 없음
- 같은 파일 내부에서만 사용 가능
- input 값을 파일 안에서 있어야 함

Script에서 Local Function 호출 예

파일 이름: `example_script.m`

```
x = 5;
square_num_1(x)

function y = square_num_1(x)
y = x^2;
end
```

Script를 실행하면 Script 안에서 함수가 호출된다.

Script 실행 과정

Command Window에서 다음과 같이 Script를 실행한다.

```
>> example_script
```

실행 과정

1. Script 코드 실행
2. Script 안에서 함수 호출
3. Local Function 실행

외부에서 Local Function 호출

Command Window에서 다음을 실행하면

```
>> square_num_1(5)
```

에러가 발생한다.

- Local Function은 같은 파일 내부에서만 사용 가능
- 다른 Script나 Command Window에서는 호출할 수 없음

외부에서 사용하려면

함수를 별도의 파일로 만들어야 한다.

파일 이름: `square_num.m`

```
function y = square_num(x)
y = x^2;
end
```

이 경우 다음과 같이 어디서든 사용할 수 있다.

```
>> square_num(5)
```

Script와 Function의 차이

구분	Script	Function
입력값	보통 없음	있음
출력값	경우에 따라 다름	있음
재사용성	보통	높음
독립성	낮음	높음
목적	작업 순서 실행	사용정의함수 사용

내장함수를 활용한 간단한 그래프 예제

```
clc; clear;
x = -10:0.1:10; % x 값 생성
y = x.^2;  % 함수 정의

disp('First five values of y:')
disp(y(1:5))  % 결과 출력

figure
plot(x,y,'LineWidth',2) % 그래프 그리기

xlabel('x')
ylabel('y = x.^2')
title('Parabola Graph')
grid on
```

작업 폴더와 도움말

```
pwd (현재 작업 폴더 확인)
dir (폴더 안 파일 목록 보기)
cd foldername (작업 폴더 변경)
help plot (간단한 도움말)
doc plot (자세한 문서)
lookfor polynomial (키워드로 함수 검색)
```

- 현재 폴더를 확인하고 파일을 관리할 수 있다.
- `help`, `doc`을 사용하면 새로운 명령을 빠르게 익힐 수 있다.